

Unidad 6 - Actividad 1

Comisión: K 3.1 Año: 2026

Grupo: 6

Integrantes:

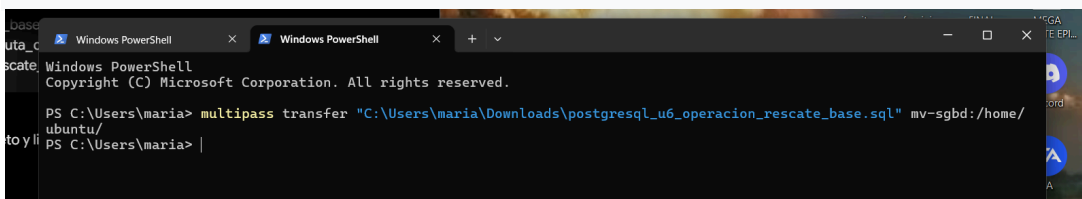
- Basualdo Alvarez, Facundo F.
- Godoy, Marianela Yasmin.
- Pare, Collante Agustina.
- Sanchez, Maria Eugenia.
- Sanchez, Romina.
- Toledo, Francisco.

Docentes:

- Prof. Andrés Fantin.
- JTP. Fernández, Juan Carlos.
- Aux. Edgar Nahuel Dominguez.

Fase 1: El usuario infiltrado

1- Ejecuten el script base del motor asignado.



Paso 1: Paso los scripts

Paso 2: Ejecutamos el script en PostgreSQL para crear la base de datos de rescate y los roles vulnerables y se obtuvo esto:

```

ubuntu@mv-sgbd:~$ sudo -u postgres psql -f /tmp/postgresql_u6_operacion_rescate_base.sql
psql:/tmp/postgresql_u6_operacion_rescate_base.sql:4: NOTICE: no existe la base de datos «u6_rescate», omitiendo
DROP DATABASE
CREATE DATABASE
Ahora está conectado a la base de datos «u6_rescate» con el usuario «postgres».
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
INSERT 0 4
INSERT 0 4
INSERT 0 5
INSERT 0 1
DO
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT
GRANT ROLE
GRANT ROLE
GRANT ROLE
clientes
-----
      4
(1 fila)

cuentas
-----
      4
(1 fila)

movimientos
-----
      5
(1 fila)

importe_movimientos
-----
73000.00
(1 fila)
ubuntu@mv-sgbd:~$ |

```

En la imagen se puede ver que se creó la base de datos u6_rescate, se estructuraron las tablas, se insertaron los datos iniciales y se configuraron los roles de laboratorio mostrando las validaciones correspondientes (4 clientes, 4 cuentas, 5 movimientos y un importe total de 73000.00).

2- Identifiquen los roles o permisos peligrosos existentes.

Paso 1: Entrar a la consola de la base de datos

Para empezar a auditar y probar, hay que conectarse a la base de datos u6_rescate como administrador ejecutando:

```

ubuntu@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2))
Digite «help» para obtener ayuda.

u6_rescate=# |

```

Paso 2: Identificar el problema (Los permisos excesivos)

Revisando el script que ejecutamos, vimos que la cátedra configuró intencionalmente una situación vulnerable: al rol **rol_app** (que usa el usuario u6_app) se le dio **permiso de TRUNCATE, DELETE, UPDATE**, etc., sobre todas las tablas, cuando una aplicación operativa no debería tener permisos destructivos totales sin control, o el analista tiene permisos que exceden su función.

3- Demuestren al menos tres accesos indebidos antes de corregirlos.

Acceso indebido N°1: Intento de borrado en la tabla del cliente

Para cumplir con este punto, demostramos que un usuario puede realizar acciones no deseadas antes de la corrección.

- A. Salimos de la consola de postgres (\q)
- B. Probamos conectándonos con el usuario de aplicación o analista para ver qué puede hacer.
- C. Por ejemplo, ingresamos con el usuario u6_app con la contraseña es Cambiar_2026!

```
u6_rescate-# \q
ubuntu@mv-sgbd:~$ |
```

Obtuvimos esto:

```
u6_rescate-# psql -U u6_app -d u6_rescate -h localhost
u6_rescate-# \q
ubuntu@mv-sgbd:~$ psql -U u6_app -d u6_rescate -h localhost
Contraseña para usuario u6_app:
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.
u6_rescate=> |
```

Como podemos ver nos conectamos a la base de datos u6_rescate utilizando el usuario de aplicación **u6_app**.

El prompt cambió a u6_rescate=>, la flecha indica que estamos operando con un usuario sin privilegios de superadministrador.

Realizamos las pruebas con u6_app

Probamos ejecutar una acción destructiva o un comando que demuestre los permisos excesivos iniciales por ejemplo ejecutamos: DELETE FROM cliente WHERE cliente_id = 4;

```
u6_rescate=> DELETE FROM cliente WHERE cliente_id = 4;
ERROR: update o delete en «cliente» viola la llave foránea «cuenta_cliente_id_fkey» en la tabla «cuenta»
DETALLE: La llave (cliente_id)=(4) todavía es referida desde la tabla «cuenta».
u6_rescate=> |
```

El motor de base de datos **sí dejó intentar la acción** (demostrando que el usuario u6_app *tiene* el privilegio de borrar), pero la base rechazó el borrado porque el cliente 4 tiene cuentas asociadas en la tabla cuenta y la regla de la base de datos protege esa relación.

Para demostrar el acceso indebido (permiso excesivo) sin que la clave foránea nos bloquee, vamos a ejecutar un DELETE sobre la tabla movimiento, ya que es una tabla transaccional que no tiene otras tablas dependiendo de ella.

Acceso indebido N.º2 : Borrado exitoso en la tabla movimientos

Comando `DELETE FROM movimiento WHERE movimiento_id = 1005;`

```
u6_rescate=> DELETE FROM movimiento WHERE movimiento_id = 1005;
DELETE 1
u6_rescate=> |
```

Retorna DELETE 1 sin errores. Esto demuestra que la aplicación puede borrar registros históricos libremente, evidenciando el riesgo de permisos excesivos que exige la **Fase 1**.

Acceso indebido N.º3 : Cambio masivo

Paso 1: saliendo de la sesión usuario con `\q`

```
u6_rescate=> \q
ubuntu@mv-sgbd:~$ |
```

paso 2: nos conectamos

```
ubuntu@mv-sgbd:~$ psql -U u6_app -d u6_rescate -h localhost
Contraseña para usuario u6_app:
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.
u6_rescate=> |
```

Paso 3: ejecutamos la prueba del cambio masivo

```
u6_rescate=> UPDATE cuenta SET saldo = 0;
UPDATE 4
u6_rescate=> |
```

Esto demuestra que el usuario de la aplicación tenía un privilegio excesivo y descontrolado que le permitió poner en cero el saldo de todas las cuentas de la base de datos de manera masiva.

Se vieron afectados 4 registros evidenciando el alto riesgo operativo de otorgar permisos de modificación globales sin restricciones en la capa de datos.

4- Rediseñen los permisos aplicando minimo privilegio.

Paso 1: Reingresamos como administrador

```
ubuntu@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2))
Digite «help» para obtener ayuda.

u6_rescate=# |
```

Paso 2: Revocamos los permisos excesivos y otorgamos solo los necesarios

Paso 3: Ejecutando el comando en SQL para corregir permisos

```
-- 1. Revocar los permisos excesivos iniciales en el esquema public
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM rol_app, rol_analista;

-- 2. Asignar los permisos estrictos para la aplicación (rol_app)
GRANT SELECT, INSERT, UPDATE ON cliente, cuenta, movimiento TO rol_app;
GRANT SELECT ON auditoria_evento TO rol_app;

-- 3. Asignar los permisos estrictos para el analista (rol_analista)
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_analista;
```

```
u6_rescate=# -- 1. Revocar los permisos excesivos iniciales en el esquema public
REVOKE ALL PRIVILEGES ON ALL TABLES IN SCHEMA public FROM rol_app, rol_analista;

-- 2. Asignar los permisos estrictos para la aplicación (rol_app)
GRANT SELECT, INSERT, UPDATE ON cliente, cuenta, movimiento TO rol_app;
GRANT SELECT ON auditoria_evento TO rol_app;

-- 3. Asignar los permisos estrictos para el analista (rol_analista)
GRANT SELECT ON ALL TABLES IN SCHEMA public TO rol_analista;
REVOKE
GRANT
GRANT
GRANT
u6_rescate=# |
```

Con esto, rol_app pierde la capacidad de DELETE/TRUNCATE sobre todas las tablas y queda limitado a SELECT, INSERT y UPDATE sobre las tablas operativas, más lectura de auditoría. rol_analista queda limitado exclusivamente a lectura (SELECT) sobre todo el esquema.

5- Vuelvan a ejecutar las pruebas y registren que operaciones quedan permitidas y cuales son rechazadas.

Entramos con el usuario `u6_analista` (contraseña `Cambiar_2026!`) e intentamos hacer un borrado o inserción para demostrar que el sistema ahora **bloquea** el intento indebido arrojando un error de permisos.

```
u6_rescate=# \q
ubuntu@mv-sgbd:~$ psql -U u6_analista -d u6_rescate -h localhost
Contraseña para usuario u6_analista:
psql (18.6 (Ubuntu 18.6-1.pgdg24.04+2))
Conexión SSL (protocolo: TLSv1.3, cifrado: TLS_AES_256_GCM_SHA384, compresión: desactivado, ALPN: postgresql)
Digite «help» para obtener ayuda.

u6_rescate=> |
```

intentamos hacer un `DELETE` o un `INSERT` para comprobar que el sistema bloquea los privilegios excesivos:

```
u6_rescate=> DELETE FROM cliente WHERE cliente_id = 1;
ERROR:  permiso denegado a la tabla cliente
u6_rescate=> |
```

Ese `ERROR` demuestra que el rol del analista ya no podemos hacer operaciones destructivas y que aplicamos correctamente el principio de mínimo privilegio.

Registro de operaciones permitidas y rechazadas:

Rol / usuario	Operación	Antes de corregir	Después de corregir
<u>rol_app</u> (<u>u6_app</u>)	DELETE en cliente	Permitido a nivel de permiso (bloqueado luego por FK)	Rechazado (sin permiso)
<u>rol_app</u> (<u>u6_app</u>)	DELETE en movimiento	Permitido y ejecutado (DELETE 1)	Rechazado (sin permiso)
<u>rol_app</u> (<u>u6_app</u>)	SELECT / INSERT / UPDATE en cliente, cuenta, movimiento	Permitido	Permitido
<u>rol_app</u> (<u>u6_app</u>)	SELECT en <u>auditoria_evento</u>	Permitido	Permitido
<u>rol_analista</u> (<u>u6_analista</u>)	DELETE / INSERT / UPDATE en cualquier tabla	Permitido (excesivo)	Rechazado (permiso denegado)
<u>rol_analista</u> (<u>u6_analista</u>)	SELECT en todas las tablas	Permitido	Permitido

6- Conclusión

Se comprobó que la configuración inicial otorgaba permisos excesivos a los roles `rol_app` y `rol_analista`, permitiendo operaciones destructivas fuera de su función. Tras aplicar `REVOKE` y `GRANT` específicos según el principio de mínimo privilegio, las pruebas posteriores confirman que las operaciones no autorizadas son rechazadas por el motor, mientras que las operaciones legítimas de cada rol continúan permitidas.

7 - Matriz de permisos

tabla	Permisos asignados a rol_app	Permisos asignados a: rol_analista
cliente	SELECT, INSERT, UPDATE	SELECT
cuenta	SELECT, INSERT, UPDATE	SELECT
movimiento	SELECT, INSERT, UPDATE	SELECT
auditoria_evento	SELECT	SELECT

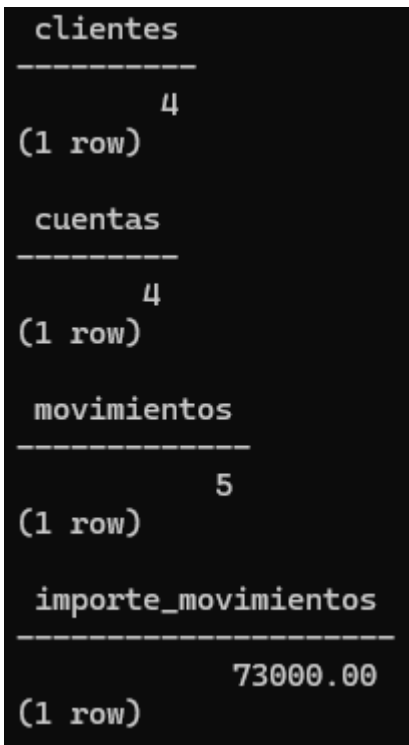
FASE 2: El DBA borro producción

1.Registrar el estado inicial de la base de datos

Antes de realizar cualquier copia, debes registrar el conteo inicial de registros y la suma de importes tal como exige la consigna. Ejecutamos el siguiente comando en la terminal:

Bash

```
sudo -u postgres psql -d u6_rescate -c "SELECT COUNT(*) AS clientes FROM cliente;  
SELECT COUNT(*) AS cuentas FROM cuenta; SELECT COUNT(*) AS movimientos FROM  
movimiento; SELECT COALESCE(SUM(importe),0) AS importe_movimientos FROM  
movimiento;"
```



```
clientes  
-----  
              4  
(1 row)  
  
cuentas  
-----  
              4  
(1 row)  
  
movimientos  
-----  
              5  
(1 row)  
  
importe_movimientos  
-----  
              73000.00  
(1 row)
```

Retornó los valores iniciales (4 clientes, 4 cuentas, 5 movimientos y 73000.00 de importe).

2. Configuración del archivo WAL antes del Backup

-Antes de realizar cualquier Backup, creamos un directorio temporal donde se guardará una copia de cada log de transacciones.

```
fran@mv-sgbd:~$ sudo mkdir -p /tmp/wal_archive
[sudo] password for fran:
fran@mv-sgbd:~$ sudo chow postgres:postgres /tmp/wal_archive
sudo: chow: command not found
fran@mv-sgbd:~$ sudo chown postgres:postgres /tmp/wal_archive
```

Bash

```
sudo mkdir -p /tmp/wal_archive
```

```
sudo chown postgres:postgres /tmp/wal_archive
```

-Con “sudo mkdir -p /tmp/wal_archive ” se crea el directorio para los archivos wal

-Con “sudo chown postgres:postgres /tmp/wal_archive ” se da permisos a postgres

Luego se ejecutó el comando: “sudo nano /etc/postgresql/16/main/postgresql.conf ”

Y se hicieron los siguientes cambios

```
wal_level = replica
```

```
archive_mode = on
```

```
archive_command = 'cp %p /tmp/wal_archive/%f'
```

-Inmediatamente despues se reinicio el servicio:

Bash:

```
sudo systemctl restart postgresql
```

```
fran@mv-sgbd:~$ sudo systemctl restart postgresql
```

3.Realizar el respaldo base

Se realiza una copia física exacta de los archivos de la Base de datos en una nueva carpeta temporal y se definen los permisos correspondientes

Bash

```
sudo mkdir -p /tmp/backup_base
```

```
sudo chown postgres:postgres /tmp/backup_base
```

```
fran@mv-sgbd:~$ sudo mkdir -p /tmp/backup_base
[sudo] password for fran:
fran@mv-sgbd:~$ sudo chown postgres:postgres /tmp/backup_base
```

Y Se ejecuta `sudo -u postgres pg_basebackup -D /tmp/backup_base -Fp -Xs -P`

```
fran@mv-sgbd:~$ sudo -u postgres pg_basebackup -D /tmp/backup_base -Fp -Xs -P
178862/178862 kB (100%), 1/1 tablespace
```

4. Ejecutar las operaciones válidas posteriores al backup

Simulamos la operatoria normal ejecutando los movimientos válidos que ocurrieron después del backup (estos datos están comentados al final del script base provisto por la cátedra):

Bash

```
sudo -u postgres psql -d u6_rescate -c "INSERT INTO movimiento VALUES
(1006,101,'2026-09-22 10:05:00','CREDITO',15000.00,'Operacion valida posterior al backup');
INSERT INTO movimiento VALUES (1007,102,'2026-09-22
10:12:00','DEBITO',-7000.00,'Operacion valida posterior al backup'); INSERT INTO
auditoria_evento(evento, detalle) VALUES ('OPERACIONES_VALIDAS', 'Se ejecutaron
movimientos 1006 y 1007');"
```

```
fran@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate -c"INSERT INTO movimiento VALUES (1006,101,'2026-09-22 10:05:00','CREDITO',15000.00,'Operacion v
alida posterior al backup');
INSERT INTO movimiento VALUES (1007,102,'2026-09-22 10:12:00','DEBITO',-7000.00,'Operacion valida posterior al backup');
INSERT INTO auditoria_evento(evento, detalle) VALUES ('OPERACIONES_VALIDAS', 'Se ejecutaron movimientos 1006 y 1007');"
INSERT 0 1
INSERT 0 1
INSERT 0 1
```

5. Simular el incidente de borrado accidental:

-Ejecutamos el comando del incidente destructivo (simulando el error del DBA que borra los movimientos de la cuenta 101):

Bash

```
sudo -u postgres psql -d u6_rescate -c "DELETE FROM movimiento WHERE cuenta_id = 101;
INSERT INTO auditoria_evento(evento, detalle) VALUES ('INCIDENTE', 'Borrado accidental de
movimientos de cuenta 101');"
```

Verificación: Si consultamos la tabla movimiento luego de estas sentencias deberíamos ver que los registros de la cuenta 101 ya no están.

```
fran@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate -c "DELETE FROM movimiento WHERE cuenta_id = 101; INSERT INTO auditoria_evento(evento, detalle) VALUES ('INCIDENTE', 'Borrado accidental de movimientos de cuenta 101');"
DELETE 3
INSERT 0 1
fran@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate -c "select from movimiento where cuenta_id = 101;"
--
(0 rows)
```

6. Preparar la recuperación

Antes de preparar la recuperación de datos, vamos a hacer uso de la tabla de auditoría para saber en qué momento ocurrió el incidente. Por el script que simula el accidente sabemos que además de ejecutar la sentencia DELETE sobre la tabla movimiento, también ejecuta una operación INSERT INTO en la tabla auditoria_evento (simulando una funcion Trigger), dicha tabla también guarda el tiempo exacto en el que se ejecutó la operación accidental. Por lo tanto se ejecutara la siguiente sentencia:

```
sudo -u postgres psql -d u6_rescate -c "select * from auditoria_evento;"
```

y se busca el registro que define al incidente:

```
fran@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate -c "select * from auditoria_evento;"
auditoria_id | fecha | usuario | evento | detalle
-----|-----|-----|-----|-----
1 | 2026-09-23 16:26:14.085102+00 | postgres | SETUP | Base inicial creada para Unidad 6
2 | 2026-09-23 17:24:03.732591+00 | postgres | OPERACIONES_VALIDAS | Se ejecutaron movimientos 1006 y 1007
3 | 2026-09-23 17:40:40.519895+00 | postgres | INCIDENTE | Borrado accidental de movimientos de cuenta 101
(3 rows)
```

Guardamos el campo fecha: "2026-09-23 17:40:40.519895+00"

-Ahora para empezar la recuperación vamos a detener momentáneamente el motor de Bases de Datos y vaciar el directorio de datos corrupto:

Bash

```
sudo systemctl stop postgresql
```

```
sudo rm -rf /var/lib/postgresql/16/main/
```

-Ya eliminados los datos "corruptos", recreamos la misma carpeta y le asignamos los permisos correspondientes

Bash

```
sudo mkdir -p /var/lib/postgresql/16/main
```

```
sudo chown postgres:postgres /var/lib/postgresql/16/main
```

-De esta manera ya tenemos devuelta nuestro directorio de datos limpio para la restauración

Verificación: Al ejecutar `sudo ls -lh /var/lib/postgresql/16/main/`, la terminal no debería mostrar ningún archivo ni carpeta.

```
fran@mv-sgbd:~$ sudo mkdir -p /var/lib/postgresql/16/main
fran@mv-sgbd:~$ sudo chown postgres:postgres /var/lib/postgresql/16/main
fran@mv-sgbd:~$ sudo ls -lh /var/lib/postgresql/16/main/
total 0
```

7. Restaurar el Backup Base

Ahora se traen los archivos limpios que se guardaron previamente al ejecutar `pg_basebackup`

Bash

```
sudo cp -R /tmp/backup_base/* /var/lib/postgresql/16/main/
```

```
sudo chown -R postgres:postgres /var/lib/postgresql/16/main/
```

Verificación: Un `ls -lh /var/lib/postgresql/16/main/` ahora debería listar múltiples carpetas (como `base`, `pg_wal`, `global`) y archivos propios del motor.

```
fran@mv-sgbd:~$ ls -lh /var/lib/postgresql/16/main/
total 380K
-rw----- 1 postgres postgres  3 Sep 27 16:45 PG_VERSION
-rw----- 1 postgres postgres 227 Sep 27 16:45 backup_label
-rw----- 1 postgres postgres 297K Sep 27 16:45 backup_manifest
drwx----- 8 postgres postgres 4.0K Sep 27 16:45 base
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 global
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_commit_ts
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_dynshmem
drwx----- 4 postgres postgres 4.0K Sep 27 16:45 pg_logical
drwx----- 4 postgres postgres 4.0K Sep 27 16:45 pg_multixact
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_notify
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_replslot
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_serial
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_snapshots
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_stat
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_stat_tmp
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_subtrans
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_tblspc
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_twophase
drwx----- 3 postgres postgres 4.0K Sep 27 16:45 pg_wal
drwx----- 2 postgres postgres 4.0K Sep 27 16:45 pg_xact
-rw----- 1 postgres postgres  88 Sep 27 16:45 postgresql.auto.conf
```

(Las fechas y horas de esta imagen no coinciden con el resto de los datos por un error al borrar la imagen anterior)

Antes de configurar el PITR creamos el archivo `recovery.signal` que le avisa al motor que debe entrar en modo de recuperación:

Bash

```
sudo -u postgres touch /var/lib/postgresql/16/main/recovery.signal
```

Y ahora editamos el archivo de configuración principal

Bash

```
sudo nano /etc/postgresql/16/main/postgresql.conf
```

Y configuramos las siguientes variables:

```
restore_command = 'cp /tmp/wal_archive/%f %p'
```

```
recovery_target_time = "2026-09-23 17:40:40.519895+00" (la hora que sacamos de la tabla de auditoría)
```

```
recovery_target_inclusive = off (el valor "off" le indica al motor detener la reproducción de logs justamente antes de la marca definida por "recovery_target_time")
```

```
recovery_target_action = 'promote' (le indica al motor que una vez alcanzado el punto objetivo de recuperación debe salir del estado de lectura y abrir la base de datos)
```

-Antes de arrancar de nuevo el servidor hay que restringir el permiso de accesibilidad a la carpeta de directorio de datos con `chmod 700`, para dar acceso solo al usuario dueño, ejecutando entonces el siguiente comando:

Bash

```
sudo chmod 700 /var/lib/postgresql/16/main
```

-La idea de aplicar este permiso es debido a que el motor de postgresql realiza una validación de seguridad al iniciar, y si detecta que existen permisos para lectura, escritura y ejecución para todos los usuarios aborta el inicio del servicio, evitando posibles ataques

-Y ahora si iniciamos el servicio

```
sudo systemctl start postgresql
```

De esta manera el motor, leerá el backup, consumirá los WAL archivados en `/tmp/wal_archive/` y construirá las transacciones válidas hasta detenerse antes del error.

Verificación: Ejecutamos `sudo -u postgres psql -d u6_rescate -c"select * from movimiento;"`. Se deberían ver los 5 movimientos originales más las operaciones válidas insertadas posteriormente, confirmando que el `DELETE` no se ejecutó.

```
fran@mv-sgbd:~$ sudo -u postgres psql -d u6_rescate -c"select * from movimiento;"
```

movimiento_id	cuenta_id	fecha	tipo	importe	descripcion
1001	101	2026-09-22 09:00:00	CREDITO	50000.00	Aporte inicial
1002	101	2026-09-22 09:10:00	DEBITO	-12000.00	Pago proveedor
1003	102	2026-09-22 09:20:00	CREDITO	30000.00	Cobranza
1004	103	2026-09-22 09:30:00	DEBITO	-5000.00	Compra
1005	104	2026-09-22 09:40:00	CREDITO	10000.00	Deposito
1006	101	2026-09-22 10:05:00	CREDITO	15000.00	Operacion valida posterior al backup
1007	102	2026-09-22 10:12:00	DEBITO	-7000.00	Operacion valida posterior al backup

(7 rows)

8. Conclusión

Se demostró con éxito la aplicación de una estrategia de recuperación ante desastres mediante PITR (*Point-In-Time Recovery*) y archivado continuo de registros de transacciones (WAL). El punto de recuperación elegido fue una **marca de tiempo exacta previa al error humano** (2026-09-23 17:40:40.519895+00, obtenida de la tabla de auditoría), porque permitió detener el motor de base de datos milisegundos antes del borrado destructivo.

Se descartó restaurar únicamente al backup inicial de la madrugada debido a que eso habría provocado la pérdida de todas las operaciones legítimas y válidas ejecutadas posteriormente (como los movimientos 1006 y 1007). Asimismo, se descartó recuperar hasta el último estado observado porque eso habría conservado y perpetuado el error del DBA en producción. De este modo, la estrategia aplicada garantizó la recuperación de la información alterada, evitando la pérdida de datos válidos y preservando de manera íntegra la operatoria posterior.

FASE 3: Diseñar para sobrevivir

Logística y Depósitos - Grupo 6

1. Arquitectura de Continuidad y Alta Disponibilidad

- Implementaremos un esquema **Activo-Pasivo** con dos servidores (nodos) utilizando réplica asíncrona en PostgreSQL.
- Esto respeta la limitación de presupuesto y hardware, permitiendo que el servidor principal atienda rápido a los depósitos sin que la copia de respaldo le reste velocidad de procesamiento.

2. Arquitectura de Protección (Estrategia de Backup y Logs)

- **Estrategia de Backup base y frecuencia:** Haremos una copia completa y exacta de toda la base de datos **todos los días a la madrugada** (por ejemplo, a las 03:00 AM), aprovechando que la operatoria logística del depósito se encuentra detenida o con actividad mínima.
- **Retención de los respaldos:** Conservaremos estos respaldos diarios por un período de **7 días** en un almacenamiento seguro para poder cubrir cualquier inconveniente retroactivo.
- **Uso de logs (Archivado de WAL):** Además del backup diario, el sistema guardará de forma ordenada y periódica los registros de cambios (*logs* de transacciones o WAL) en una ubicación externa. Esto nos permite asegurar el flujo continuo de datos recientes sin saturar ni asfixiar el disco del servidor principal.

3. Arquitectura de Recuperación (PITR y Errores)

- **Posibilidad de PITR (Point-In-Time Recovery):** Gracias a la combinación del backup diario y el historial de logs guardados, mantendremos habilitada la "máquina del tiempo". Esto significa que ante un desastre, podremos reconstruir la base de datos y detenemos en el segundo exacto previo a que ocurriera un error.
- **Tratamiento ante un borrado accidental (Error humano):** Si un operador o administrador borra datos por error, utilizaremos el respaldo base junto con los archivos de historial para hacer un PITR. Así, restauramos la información limpia justo antes del error y volvemos a aplicar o mantener las operaciones válidas que se hicieron después, sin perder datos legítimos.

4. Procedimiento ante la caída del servidor primario (Continuidad)

- **Si se cae el servidor principal:** Se aísla el equipo dañado y se promueve manualmente el servidor secundario (*Standby*) para que tome su lugar y redireccione el tráfico de la red.
- **Si hay un borrado accidental (Error humano):** Se utiliza el respaldo base junto con los archivos de historial para reconstruir la información justa hasta antes del error, evitando perder las operaciones legítimas posteriores.

5. Evidencia del cumplimiento de RPO y RTO (Métricas)

- **Evidencia del RPO (Recovery Point Objective $\leq$ 15 minutos):**
 - Como el backup base se hace diariamente y los cambios recientes se van guardando de manera constante en los registros de historial (WAL), ante una caída catastrófica la máxima información que podríamos llegar a perder es la generada en los últimos minutos previos al corte. Esto garantiza que **no superamos la ventana límite de 15 minutos** exigida por la empresa.
- **Evidencia del RTO (Recovery Time Objective $\leq$ 30 minutos):**
 - Aislar un servidor caído y activar manualmente el nodo de repuesto toma pocos minutos. Incluso si hubiera que levantar un respaldo desde cero, el proceso completo se completa en menos de 20 minutos, encajando perfectamente dentro de la **meta de media hora** para que la logística del depósito no sufra parates prolongados.

6. Costos, Riesgos y Limitaciones de la solución elegida

- **El riesgo del *Split-Brain*:** Al contar únicamente con dos nodos y un presupuesto medio, **no tenemos un tercer servidor "árbitro"** que decida automáticamente qué equipo debe mandar si se corta la red.
- **Consecuencia:** Si dejamos la conmutación totalmente automatizada, ante un corte de red ambos nodos podrían creer que son los jefes, procesando datos distintos y corrompiendo el inventario del depósito (*Split-Brain*).
- **Mitigación:** Para evitar que ambos servidores creen que son el jefe al mismo tiempo y corrompan el stock, definimos que **la conmutación o ascenso del servidor secundario requerirá siempre la validación y ejecución humana de un DBA**, priorizando la seguridad de los datos por sobre la automatización riesgosa.